



BUAP



**Facultad de
Cs. de la Computación**

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

MINERÍA DE DATOS

Implementación del Algoritmo de Clasificación K-NN

Integrantes:

Arantza Ximena Saucedo Roman

201957079

Berenice Juárez González 202042438

César Alberto Viveros Santiago 201945254

Profesor

Pedro Tecuanhuehue Vera

Facultad de Ciencias de la Computación

8 de febrero de 2024

Índice

1. INTRODUCCIÓN	3
2. DESCRIPCIÓN DEL ALGORITMO K-NN	3
3. METODOLOGÍA:	3
4. ENTRENAMIENTO DEL MODELO:	4
5. CLASIFICACIÓN DE DATOS DESCONOCIDOS:	5
6. CONCLUSIÓN:	6
7. APÉNDICES:	6
7.1. CÓDIGO COMENTADO:	6
7.2. DESCRIPCIÓN DE CONCEPTOS USADOS:	12
7.3. MANUAL DE USUARIO:	12
7.4. DETALLES TÉCNICOS:	17

1. INTRODUCCIÓN

El código proporcionado es un programa en Python que implementa el algoritmo de K-Nearest Neighbors (KNN) para la clasificación de objetos desconocidos. Utiliza la biblioteca Tkinter para crear una interfaz gráfica de usuario (GUI) que permita al usuario cargar un archivo .csv, especificar el valor de K y realizar la clasificación de objetos desconocidos. En este reporte, se analiza en detalle cada componente del código, explicando su función, de igual forma se anexa un manual de usuario para el código.

2. DESCRIPCIÓN DEL ALGORITMO K-NN

El algoritmo K-NN (K Nearest Neighbors o K Vecinos Más Cercanos) es un algoritmo de aprendizaje supervisado simple y efectivo que se utiliza tanto para clasificación como para regresión. Su funcionamiento se basa en la idea de que los puntos de datos similares se agrupan en el espacio de características.

Funcionamiento:

- 1. Preparación de datos:** Se le asigna un archivo con la base de datos a analizar.
- 2. Entrenamiento y evaluación:** El algoritmo entrena y evalúa el conjunto de datos previamente asignado (entrenamiento) con características y etiquetas utilizando un $K = 3, 5$ y 7 .
- 3. Selección de K:** Se define el valor de K, que representa la cantidad de vecinos que se considerarán para la predicción sobre un conjunto de datos desconocido.
- 4. Carga de datos desconocidos:** Se carga otro conjunto de datos desconocidos para el algoritmo.
- 5. Cálculo de la distancia:** Se calcula la distancia entre el nuevo punto de datos y todos los puntos del conjunto de entrenamiento utilizando la distancia euclidiana.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + \dots + (x_n - x_n)^2} \quad (1)$$

- 6. Selección y clasificación:** Se realiza la clasificación y predicción de los nuevos valores para cierta etiqueta.
Selección de los vecinos: Se seleccionan los K puntos del conjunto de entrenamiento con la menor distancia al nuevo punto.

Predicción:

- Clasificación: Se asigna al nuevo punto la clase mayoritaria entre los K vecinos.
- Regresión: Se calcula el valor promedio de la variable objetivo entre los K vecinos

3. METODOLOGÍA:

La metodología de este código se centra en la clasificación de objetos utilizando el algoritmo K-NN (K-Nearest Neighbors). El código sigue los siguientes pasos:

- Conocer sobre el tema: Primero tuvimos que investigar más a fondo cómo es que funciona el K-NN, obteniendo información en internet y poniéndolo en práctica pudimos comprender mejor el tema y tener una idea de cómo implementarlo en nuestro código.
- Importación de bibliotecas: Al principio, se importan las bibliotecas necesarias, como Pandas, Tkinter, para gestionar datos y crear la interfaz gráfica de usuario.

- Hicimos un programa que calcula la distancia euclidiana entre dos puntos, es decir, los datos clasificados, de igual forma el programa puede clasificar.

En el caso de los datos clasificados :

- Lee el archivo cargado (.CVS) desde la carpeta ya seleccionada.
- Clasifica todos los datos ingresados en 2 columnas (entrenamiento y prueba), cada conjunto cuenta con datos al azar y sin repetirse.
- Se ejecuta el modelo para el valor de K (3, 5 y 7) estas siendo las distancias de vecinos cercanos que se consideran.
- Evalúa el rendimiento del modelo para conocer la precisión del mismo.

En el caso de los datos desconocidos :

- Lee el archivo cargado (.CVS) desde la carpeta ya seleccionada.
- Procesa y clasifica los datos ingresados (puede hacer otro procesamiento adicional si es requerido).
- Se ejecuta el modelo para las iteraciones establecidas y se genera el rendimiento del mismo.
- Creación de interfaz: Creamos una interfaz gráfica de usuario utilizando la biblioteca Tkinter en la interfaz incluye los botones para cargar los archivos .csv, entrenar y clasificar datos desconocidos mostrando sus resultados.

En resumen, la metodología del código implica la carga de datos desde un archivo .csv, la clasificación de objetos desconocidos utilizando K-NN con diferentes valores de K, la selección del valor de K-óptimo y la presentación de resultados a través de una interfaz gráfica de usuario.

4. ENTRENAMIENTO DEL MODELO:

El entrenamiento del modelo básicamente implica almacenar los datos de entrenamiento y, durante la fase de predicción, calcula las distancias entre las instancias de entrenamiento y las nuevas instancias para encontrar los vecinos más cercanos y hacer una predicción basada en ellos.

- El código itera a través de diferentes valores de K especificados en la lista de valores para k, que sería 3, 5, 7.
- Para cada valor de K, se realiza una prueba de evaluación de 20 veces. En cada iteración, se toma una muestra aleatoria de los datos para entrenamiento y otra para prueba.
- En la división de datos creamos dos, los datos de entrenamiento y los datos de prueba, los de entrenamiento se crean seleccionando aleatoriamente 10 filas de la base de datos para usarlos como datos de entrenamiento. Los de prueba se crean seleccionando aleatoriamente 4 filas de la base de datos para usarlos como datos de prueba.
- Se calcula la distancia euclidiana para cada dato.
- Se seleccionan los K vecinos más cercanos basados en las distancias.
- Se determina la etiqueta en función de las etiquetas de sus vecinos más cercanos.
- Se calcula la precisión de clasificación para esa iteración en particular, dividiendo el número de predicciones correctas entre el número total de instancias de prueba.
- Por último se almacenan los resultados de las 20 iteraciones con su respectivo valor de K.

● Cómo se puede observar en la Figura 1 y la Figura 2 se muestran las 20 iteraciones, con su respectivo valor de K, la primera fila es de K=3, la segunda fila es K=5 y la última le corresponde a K=7. En la parte derecha se muestran los resultados de promedio del rendimiento, donde k=3 tuvo 0.57 siendo el mejor K para este problema.

i	K = 3	K = 5	K = 7
1	0.40	0.40	0.40
2	0.40	0.60	0.60
3	0.60	0.60	0.60
4	0.60	0.80	0.60
5	0.40	0.40	0.40
6	0.40	0.40	0.40
7	0.80	1.00	0.80
8	0.40	0.40	0.60
9	0.40	0.40	0.40
10	0.60	0.60	0.60
11	0.60	0.60	0.80
12	0.60	0.80	0.60
13	0.60	0.60	0.60
14	1.00	0.80	0.80
15	0.60	0.60	0.60
16	0.60	0.80	0.80
17	0.60	0.60	0.60
18	0.60	0.60	0.60

Resultados promedio de rendimiento:
 K = 3: 0.57
 K = 5: 0.61
 K = 7: 0.60

Figura 1: Datos engrenados 1-18

19	0.60	0.60	0.60
20	0.60	0.60	0.60

Figura 2: Datos entrenados 19-20

5. CLASIFICACIÓN DE DATOS DESCONOCIDOS:

La clasificación de datos desconocidos implica encontrar los k vecinos más cercanos en el conjunto de datos de entrenamiento y determinar la etiqueta de clase más común entre esos vecinos para asignarla a la instancia desconocida. Es importante tener en cuenta que el valor de k puede influir en la precisión del modelo. Un valor de k más grande puede suavizar la frontera de decisión, mientras que un valor de k más pequeño puede hacer que el modelo sea más sensible al ruido en los datos.

Le damos los datos desconocidos a nuestro programa y los probamos para $K=3$, $K=5$ y $K=7$.

Temperati	Humedad	Viento	Jugar_Gol	Soleado	Nublado	Lluvioso	Clasificación
77	92	0	0	1	0	0	No
63	78	1	0	0	0	1	Si
88	70	0	1	0	1	0	Si
72	95	0	1	0	0	1	No
79	84	1	0	0	0	1	Si
68	72	0	1	0	1	0	Si
85	88	0	0	0	1	0	No
74	90	1	0	0	0	1	No
62	75	0	1	1	0	0	Si
81	82	1	0	0	0	1	Si
73	89	0	1	0	0	1	No
67	76	0	1	0	1	0	Si

Figura 3: $K=3$

Temperatu	Humedad	Viento	Jugar_Gol	Soleado	Nublado	Lluvioso	Clasificación
77	92	0	0	1	0	0	No
63	78	1	0	0	0	1	Si
88	70	0	1	0	1	0	Si
72	95	0	1	0	0	1	No
79	84	1	0	0	0	1	Si
68	72	0	1	0	1	0	Si
85	88	0	0	0	1	0	Si
74	90	1	0	0	0	1	No
62	75	0	1	1	0	0	Si
81	82	1	0	0	0	1	Si
73	89	0	1	0	0	1	No
67	76	0	1	0	1	0	Si

Figura 4: K=5

Temperatu	Humedad	Viento	Jugar_Gol	Soleado	Nublado	Lluvioso	Clasificación
77	92	0	0	1	0	0	No
63	78	1	0	0	0	1	Si
88	70	0	1	0	1	0	Si
72	95	0	1	0	0	1	Si
79	84	1	0	0	0	1	Si
68	72	0	1	0	1	0	Si
85	88	0	0	0	1	0	Si
74	90	1	0	0	0	1	Si
62	75	0	1	1	0	0	Si
81	82	1	0	0	0	1	Si
73	89	0	1	0	0	1	Si
67	76	0	1	0	1	0	Si

Figura 5: K=7

6. CONCLUSIÓN:

El código proporcionado implementa el algoritmo K-NN para la clasificación de datos. Se utiliza un conjunto de datos de golf como ejemplo y se evalúa el rendimiento del modelo para diferentes valores de K. Este enfoque se utiliza para determinar el valor de K óptimo para el conjunto de datos en cuestión. La elección de un valor de K adecuado es crítica para el rendimiento del modelo K-NN y puede variar según el conjunto de datos. En este caso, el código identifica el valor de K que proporciona el mejor rendimiento promedio en las 20 iteraciones realizadas. En resumen, el código es un ejemplo de cómo implementar y evaluar el modelo K-NN para la clasificación de datos y puede ser adaptado para su uso en otros conjuntos de datos y problemas de clasificación.

7. APÉNDICES:

7.1. CÓDIGO COMENTADO:

```

1 import csv
2 import random
3 import math
4 import tkinter as tk
5 from tkinter import ttk, filedialog
6 import pandas as pd
7
8 def cargar_datos(nombre_archivo):
9     """
10     Carga los datos desde un archivo CSV, moviendo la cuarta columna al final.
11
12     Parametros:
13     - nombre_archivo: Ruta del archivo CSV a cargar.
14
15     Retorna:
16     - Una lista de listas con los datos cargados y modificados.
17     """
18     datos = []
19     with open(nombre_archivo, 'r') as archivo_csv:
20         lector_csv = csv.reader(archivo_csv)
21         encabezados = next(lector_csv) # Omitir encabezados si es necesario
22         for fila in lector_csv:
23             fila = list(map(float, fila))

```

```
24         cuarta_columna = fila.pop(3)
25         fila.append(cuarta_columna)
26         datos.append(fila)
27     return datos
28
29 def dividir_datos(datos, proporcion_entrenamiento):
30     """
31     Divide los datos en conjuntos de entrenamiento y prueba.
32
33     Parametros:
34     - datos: Lista de datos a dividir.
35     - proporcion_entrenamiento: Proporción del conjunto de entrenamiento.
36
37     Retorna:
38     - Tupla con dos listas: conjunto de entrenamiento y conjunto de prueba.
39     """
40     random.shuffle(datos)
41     num_muestras = len(datos)
42     num_entrenamiento = int(num_muestras * proporcion_entrenamiento)
43     return datos[:num_entrenamiento], datos[num_entrenamiento:]
44
45 def distancia_euclidiana(punto1, punto2):
46     """
47     Calcula la distancia euclidiana entre dos puntos.
48
49     Parametros:
50     - punto1: Primer punto como lista o tupla de coordenadas.
51     - punto2: Segundo punto como lista o tupla de coordenadas.
52
53     Retorna:
54     - Distancia euclidiana entre punto1 y punto2.
55     """
56     return math.sqrt(sum([(x - y) ** 2 for x, y in zip(punto1, punto2)]))
57
58 def predecir_clase(conjunto_entrenamiento, punto_prueba, K):
59     """
60     Predice la clase de un punto de prueba utilizando el algoritmo k-NN.
61
62     Parametros:
63     - conjunto_entrenamiento: Lista de puntos de entrenamiento.
64     - punto_prueba: Punto de prueba para clasificar.
65     - K: Número de vecinos más cercanos a considerar.
66
67     Retorna:
68     - La clase predicha para el punto de prueba.
69     """
70     vecinos_cercanos = sorted(conjunto_entrenamiento, key=lambda x: distancia_euclidiana(x
71         [-1], punto_prueba[-1]))[:K]
72     clases_vecinos = [vecino[-1] for vecino in vecinos_cercanos]
73     return max(set(clases_vecinos), key=clases_vecinos.count)
74
75 def evaluar_modelo(conjunto_entrenamiento, conjunto_prueba, k):
76     """
77     Evalúa el rendimiento del modelo k-NN en el conjunto de prueba.
78
79     Parametros:
80     - conjunto_entrenamiento: Lista de puntos de entrenamiento.
81     - conjunto_prueba: Lista de puntos de prueba.
82     - k: Número de vecinos más cercanos a considerar.
83
84     Retorna:
85     - Precisión del modelo como un valor flotante.
86     """
87     aciertos = sum(1 for punto_prueba in conjunto_prueba if punto_prueba[-1] == predecir_clase
88         (conjunto_entrenamiento, punto_prueba, k))
89     return aciertos / len(conjunto_prueba)
90
91 def cargar_datos_desconocidos():
```

```

91     Carga los datos desconocidos desde un archivo CSV seleccionado por el usuario.
92
93     Retorna:
94     - DataFrame de pandas con los datos desconocidos cargados.
95     """
96     ruta_archivo = filedialog.askopenfilename(filetypes=[("CSV Files", "*.csv")])
97     if ruta_archivo:
98         datos_desconocidos = pd.read_csv(ruta_archivo)
99         # Procesamiento adicional si es necesario
100         return datos_desconocidos
101     return None
102
103 def mostrar_resultados_clasificacion(resultados):
104     """
105     Muestra los resultados de clasificaci n en una nueva ventana.
106
107     Parametros:
108     - resultados: Lista de resultados de clasificaci n para mostrar.
109     """
110     ventana_resultados = tk.Toplevel()
111     ventana_resultados.title("Resultados de predicci n del partido de golf")
112     ventana_resultados.geometry("600x400")
113
114     # Crear un Frame como contenedor
115     contenedor = ttk.Frame(ventana_resultados)
116     contenedor.pack(fill=tk.BOTH, expand=True)
117
118     # Crear el Treeview dentro del contenedor
119     tabla_resultados = ttk.Treeview(contenedor, columns=("Clasificaci n de predicci n"),
120                                     show="headings")
121     tabla_resultados.heading("#1", text="Clasificaci n")
122     tabla_resultados.column("#1", anchor=tk.CENTER)
123
124     # Insertar los resultados en el Treeview
125     for i, resultado in enumerate(resultados):
126         clasificacion = "S " if resultado == 1 else "No" # Cambiar 1 a "S " y 0 a "No"
127         tabla_resultados.insert("", tk.END, values=(clasificacion,))
128
129     # Crear Scrollbar y asociarlo con el Treeview
130     scrollbar = ttk.Scrollbar(contenedor, orient=tk.VERTICAL, command=tabla_resultados.yview)
131     tabla_resultados.config(yscrollcommand=scrollbar.set)
132
133     # Colocar el Treeview y el Scrollbar en el contenedor
134     tabla_resultados.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
135     scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
136
137 # Funci n para mostrar las 20 iteraciones en una ventana con los resultados promedio de
138 # rendimiento
139 def mostrar_iteraciones(iteraciones, promedios_rendimiento):
140     """
141     Muestra las iteraciones y los resultados promedio de rendimiento en una nueva ventana.
142
143     Parametros:
144     - iteraciones: Lista de iteraciones para mostrar.
145     - promedios_rendimiento: Diccionario con los promedios de rendimiento.
146     """
147     ventana_iteraciones = tk.Toplevel()
148     ventana_iteraciones.title("20 Iteraciones y Resultados Promedio de Rendimiento")
149     ventana_iteraciones.geometry("900x700") # Establecer tama o personalizado
150     ventana_iteraciones.config(bg='#ADD8E6') # Fondo azul claro para la ventana
151
152     # Configurar estilo para los widgets dentro de esta ventana
153     estilo = ttk.Style()
154     estilo.configure('TFrame', background='#ADD8E6')
155     estilo.configure('TLabel', font=('Helvetica', 16), background='#ADD8E6', foreground='black')
156     estilo.configure('TScrollbar', background='#ADD8E6')
157
158     # Crear un marco para los resultados promedio a la derecha

```



```

157 frame_resultados = ttk.Frame(ventana_iteraciones, style='TFrame')
158 frame_resultados.pack(side="right", padx=10, pady=10, fill="y", expand=False)
159
160 etiqueta_resultados = ttk.Label(frame_resultados, text="Resultados promedio de rendimiento
: ", font=("Helvetica", 14, 'bold'), background='#ADD8E6', foreground='black')
161 etiqueta_resultados.pack()
162
163 for k, rendimiento_promedio in promedios_rendimiento.items():
164     etiqueta = ttk.Label(frame_resultados, text=f'{k}: {rendimiento_promedio:.2f}', font=(
"Helvetica", 16), background='#ADD8E6', foreground='black')
165     etiqueta.pack(anchor="e")
166
167 # Crear un widget Text para las iteraciones con un Scrollbar para el desplazamiento
168 text_iteraciones = tk.Text(ventana_iteraciones, font=("Helvetica", 15), wrap="word",
height=10, bd=0, padx=10, pady=10, bg="#E1E1E1", fg="black")
169 scrollbar_iteraciones = ttk.Scrollbar(ventana_iteraciones, orient="vertical", command=
text_iteraciones.yview, style='TScrollbar')
170 text_iteraciones.configure(yscrollcommand=scrollbar_iteraciones.set)
171
172 # Insertar texto de iteraciones en el widget Text
173 text_iteraciones.insert(tk.END, "\n".join(iteraciones))
174 text_iteraciones.config(state=tk.DISABLED) # Deshabilitar edici n
175
176 # Empaquetar el widget Text y el Scrollbar
177 text_iteraciones.pack(side="left", fill="both", expand=True)
178 scrollbar_iteraciones.pack(side="left", fill="y")
179
180
181 # Funci n para manejar la clasificaci n y mostrar los resultados en la GUI
182 def clasificar_datos(conjunto_entrenamiento, k_entry):
183     """
184     Maneja la clasificaci n de datos desconocidos y muestra los resultados.
185
186     Par metros:
187     - conjunto_entrenamiento: Lista de puntos de entrenamiento.
188     - k_combobox: Widget Combobox de donde se obtiene el valor de K seleccionado.
189     """
190     datos_desconocidos = cargar_datos_desconocidos()
191     if datos_desconocidos is not None:
192         resultado = []
193
194         k = int(k_entry.get()) # Obtiene el valor de K ingresado manualmente
195         for _, fila in datos_desconocidos.iterrows():
196             punto_prueba = fila.values[:-1] # Excluye la ltima columna (la clase)
197             clase_predicha = predecir_clase(conjunto_entrenamiento, punto_prueba, k)
198             resultado.append(clase_predicha)
199             mostrar_resultados_clasificacion(resultado)
200
201
202
203 def mostrar_archivo_csv(ruta_archivo):
204     """
205     Abre una nueva ventana para mostrar el contenido de un archivo CSV en formato de tabla.
206
207     Par metros:
208     - ruta_archivo: Ruta del archivo CSV a mostrar.
209     """
210
211     ventana_tabla = tk.Toplevel()
212     ventana_tabla.title("Vista de Archivo CSV")
213     ventana_tabla.geometry("800x600")
214
215     # Configurar el estilo de los widgets
216     estilo = ttk.Style()
217     # Guardar la configuraci n original del estilo
218     original_font = estilo.lookup("Treeview", "font")
219     original_rowheight = estilo.lookup("Treeview", "rowheight")
220
221     # Configurar el estilo de los widgets para esta ventana

```

```

222     estilo.configure("Treeview",
223                     background="#D3D3D3",
224                     foreground="black",
225                     rowheight=25, # Aumenta el rowheight si es necesario para acomodar el
                                     tama o de la fuente
226                     fieldbackground="#D3D3D3",
227                     font=('Arial', 14)) # Aumenta el tama o de la fuente aqu
228
229     estilo.map('Treeview', background=[('selected', '#347083')])
230
231 # Estilo para el Treeview Heading (cabecera)
232     estilo.configure("Treeview.Heading",
233                     font=('Arial', 16, 'bold'), # Ajusta la fuente para los encabezados
234                     background="#8B0000",
235                     foreground="white")
236     estilo.map('Treeview.Heading', background=[('active', '#B22222')])
237
238 # Leer los datos del archivo CSV
239     try:
240         datos = pd.read_csv(ruta_archivo)
241     except Exception as e:
242         print("Error al leer el archivo CSV:", e)
243         return
244
245 # Crear el Treeview con Scrollbar
246     frame = ttk.Frame(ventana_tabla)
247     frame.pack(fill=tk.BOTH, expand=True)
248
249     tabla = ttk.Treeview(frame, columns=list(datos.columns), show="headings", style="Treeview"
250                          )
251     tabla.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
252
253     scrollbar = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=tabla.yview)
254     tabla.configure(yscrollcommand=scrollbar.set)
255     scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
256
257     for columna in datos.columns:
258         tabla.heading(columna, text=columna)
259         tabla.column(columna, anchor=tk.CENTER)
260
261 # Insertar los datos en el Treeview
262     for _, fila in datos.iterrows():
263         tabla.insert("", tk.END, values=tuple(fila))
264
265     def on_closing():
266         """Restablece el estilo original cuando la ventana se cierra."""
267         estilo.configure("Treeview", font=original_font, rowheight=original_rowheight)
268         ventana_tabla.destroy()
269
270     ventana_tabla.protocol("WM_DELETE_WINDOW", on_closing)
271
272 # Funci n principal
273     def main():
274         """
275         Funci n principal que inicia la aplicaci n GUI.
276         """
277         nombre_archivo = 'archivo_modificado.csv'
278         datos = cargar_datos(nombre_archivo)
279         num_iteraciones = 20
280         proporcion_entrenamiento = 0.714 # 10 / 14 (tama o del conjunto de entrenamiento /
281                                           tama o total)
282         promedios_rendimiento = {'K = 3': 0, 'K = 5': 0, 'K = 7': 0}
283         iteraciones = []
284
285         resultados_por_iteracion = []
286
287         for _ in range(num_iteraciones):

```

```

288     rendimientos = {}
289     conjunto_entrenamiento, conjunto_prueba = dividir_datos(datos,
290         proporcion_entrenamiento)
291     for k in [3, 5, 7]:
292         rendimiento = evaluar_modelo(conjunto_entrenamiento, conjunto_prueba, k)
293         rendimientos[f'K = {k}'] = rendimiento
294         promedios_rendimiento[f'K = {k}'] += rendimiento / num_iteraciones
295
296     resultados_por_iteracion.append(rendimientos)
297
298     mejor_k = max(promedios_rendimiento, key=promedios_rendimiento.get)
299     mejor_promedio = promedios_rendimiento[mejor_k]
300
301     iteraciones.append("\n\tK = 3 \t\tK = 5 \t\tK = 7 ")
302     for i, resultados in enumerate(resultados_por_iteracion, start=1):
303         rendimientos_i = [resultados[f'K = {k}'] for k in [3, 5, 7]]
304         iteraciones.append(f"\n{i}\t\t{rendimientos_i[0]:.2f}\t\t{rendimientos_i[1]:.2f}\t\t{rendimientos_i[2]:.2f}")
305
306     ventana_principal = tk.Tk()
307     ventana_principal.title("Clasificación K-NN")
308     ventana_principal.geometry("700x500") # Ajustado para una mejor visualización
309     ventana_principal.configure(bg='#ADD8E6') # Fondo azul claro
310
311     # Configuración del estilo para los widgets
312     estilo = ttk.Style()
313     estilo.theme_use('clam') # Usar un tema que permita personalización
314     estilo.configure('TLabel', font=('Helvetica', 14), background='#ADD8E6')
315     estilo.configure('TButton', font=('Helvetica', 14), padding=10)
316     estilo.configure('TCombobox', font=('Helvetica', 14))
317     ventana_principal.option_add('*TCombobox*Listbox.font', ('Helvetica', 14)) # Ajusta el tamaño de la fuente de los items del Combobox
318
319     etiqueta_principal = ttk.Label(ventana_principal, text='Implementación del Algoritmo KNN',
320         font=('Helvetica', 18), background='#ADD8E6')
321     etiqueta_principal.pack(pady=20)
322
323     etiqueta_mejor_k = ttk.Label(ventana_principal, text=f'El mejor promedio fue {mejor_k} = {mejor_promedio:.2f}',
324         font=('Helvetica', 16), background='#ADD8E6')
325     etiqueta_mejor_k.pack(pady=10)
326
327     boton_cargar_datos = ttk.Button(ventana_principal, text="Cargar Datos a Clasificar",
328         command=lambda: clasificar_datos(datos, k_entry), style='TButton')
329     boton_cargar_datos.pack(pady=10)
330
331     boton_mostrar_iteraciones = ttk.Button(ventana_principal, text="Mostrar las 20 iteraciones",
332         command=lambda: mostrar_iteraciones(iteraciones, promedios_rendimiento), style='TButton')
333     boton_mostrar_iteraciones.pack(pady=10)
334
335     etiqueta_k = ttk.Label(ventana_principal, text="Ingresa el valor de K: \n (Solo 3, 5 o 7)",
336         font=('Helvetica', 14), background='#ADD8E6')
337     etiqueta_k.pack(pady=10)
338
339     k_entry = ttk.Entry(ventana_principal, font=('Helvetica', 14))
340     k_entry.pack(pady=10)
341
342     boton_abrir_modificado = ttk.Button(ventana_principal, text="Abrir Archivo Modificado",
343         command=lambda: mostrar_archivo_csv('archivo_modificado.csv'))
344     boton_abrir_modificado.pack(pady=10)
345
346     boton_abrir_desconocidos = ttk.Button(ventana_principal, text="Abrir Datos Desconocidos",
347         command=lambda: mostrar_archivo_csv('Desconocidos.csv'))
348     boton_abrir_desconocidos.pack(pady=10)
349
350     ventana_principal.mainloop()
351

```

```
346 if __name__ == "__main__":  
347     main()
```

7.2. DESCRIPCIÓN DE CONCEPTOS USADOS:

● **Distancia Euclidiana:** La distancia euclidiana es una métrica que se utiliza para calcular la distancia entre dos puntos en un espacio euclidiano (como un plano o un espacio tridimensional). En el contexto del código, se utiliza para medir la similitud entre dos conjuntos de características (vectores) calculando la longitud del segmento de línea recta que los conecta.

En su forma más básica, la fórmula de la distancia euclidiana se expresa como:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + \dots + (x_n - x_n)^2} \quad (2)$$

Donde **x** y **y** son los dos puntos y **n** es el número de dimensiones (atributos).

● **Biblioteca Pandas y Tkinter:** Pandas es una biblioteca de Python utilizada para analizar datos, Tkinter es la interfaz estándar de Python para el kit de herramientas de interfaz gráfica de usuario, permite crear interfaces para sus aplicaciones de Python.

● **Algoritmo K-Nearest Neighbors (KNN):** KNN es un algoritmo de aprendizaje supervisado simple y efectivo utilizado tanto para clasificación como para regresión. Se basa en la idea de que los puntos de datos similares se agrupan en el espacio de características. Entonces, para predecir la clase o valor de un nuevo punto de datos, podemos mirar sus 'vecinos más cercanos' en el conjunto de datos de entrenamiento.

● **Regresión:** Se refiere a la tarea de predecir un valor numérico continuo a partir de un conjunto de variables de entrada (características). El objetivo es encontrar una función que modele la relación entre las características y la variable objetivo.

● **Clasificación:** Es la tarea de asignar un nuevo punto de datos a una categoría específica a partir de un conjunto de variables de entrada (características). El objetivo es encontrar un modelo que pueda distinguir entre diferentes categorías con la mayor precisión posible.

7.3. MANUAL DE USUARIO:

Descripción:

Este manual describe cómo utilizar un algoritmo K-NN (vecinos más cercanos) implementado en Python para realizar la clasificación de datos. El algoritmo K-NN es un método de aprendizaje supervisado utilizado para clasificar puntos de datos en función de las características de los vecinos más cercanos en un conjunto de datos de entrenamiento.

Requisitos:

- Python 3.x instalado en su sistema.
- Instalación de las bibliotecas necesarias: Tkinter, Pandas.

Uso del Código:

1. Descargar los archivos:

- Descargue la carpeta con nombre Minería que contiene el código y los archivos .csv desde el repositorio correspondiente y contendrá los siguientes archivos.

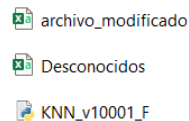


Figura 6: Carpeta Minería.

2. Instalar Bibliotecas:

- Abra una terminal o línea de comandos, navegue hasta el directorio dónde se encuentra el código fuente y ejecute el siguiente comando para instalar las dependencias necesarias:

```
PS C:\Users\bjuar\OneDrive\Documentos\7 semestre\mineria de datos\mineria> pip install pandas
```

Figura 7: Biblioteca Pandas.

```
PS C:\Users\bjuar\OneDrive\Documentos\7 semestre\mineria de datos\mineria> pip install tk
```

Figura 8: Biblioteca Tkinter.

3. Preparar los Datos:

- Verifique que los datos de entrenamiento y prueba estén en sus respectivos archivos .csv. Asegure de que los datos estén formateados correctamente.
- Los datos de entrenamiento y prueba deben estar en archivos .csv separados como se muestra en la Figura 6.

4. Preparar el Código:

- Primero abriremos nuestro entorno de desarrollo preferido, en este caso nosotros usaremos Spyder.
- Una vez que el programa esté abierto, nos dirigimos a la parte superior dónde dice Archivo, seleccionamos Abrir y escogemos nuestro código en Python.
- Nos mostrará el siguiente código Figura 9.

```

KNN_v10001_F.py X
1 import csv
2 import random
3 import math
4 import tkinter as tk
5 from tkinter import ttk, filedialog
6 import pandas as pd
7
8 def cargar_datos(nombre_archivo):
9     """
10     Carga los datos desde un archivo CSV, moviendo la cuarta columna al final.
11
12     Parámetros:
13     - nombre_archivo: Ruta del archivo CSV a cargar.
14
15     Retorna:
16     - Una lista de listas con los datos cargados y modificados.
17     """
18     datos = []
19     with open(nombre_archivo, 'r') as archivo_csv:
20         lector_csv = csv.reader(archivo_csv)
21         encabezados = next(lector_csv) # Omitir encabezados si es necesario
22         for fila in lector_csv:
23             fila = list(map(float, fila))
24             cuarta_columna = fila.pop(3)
25             fila.append(cuarta_columna)
26             datos.append(fila)
27     return datos
28
29 def dividir_datos(datos, proporcion_entrenamiento):
30     """
31     Divide los datos en conjuntos de entrenamiento y prueba.
32
33     Parámetros:
34     - datos: Lista de datos a dividir.
35     - proporcion_entrenamiento: Proporción del conjunto de entrenamiento.
36
37     Retorna:
38     - Tupla con dos listas: conjunto de entrenamiento y conjunto de prueba.
39     """
40     random.shuffle(datos)
41     num_muestras = len(datos)
42     num_entrenamiento = int(num_muestras * proporcion_entrenamiento)
43     return datos[:num_entrenamiento], datos[num_entrenamiento:]
44
45 def distancia_euclidiana(punto1, punto2):

```

Figura 9: Abrir Archivo.

5. Ejecución del código.

- En la parte superior de nuestra ventana encontraremos el siguiente ícono.
- Le damos clic.



Figura 10: Ícono de ejecutar.

- Nos mostrará la siguiente ventana principal.

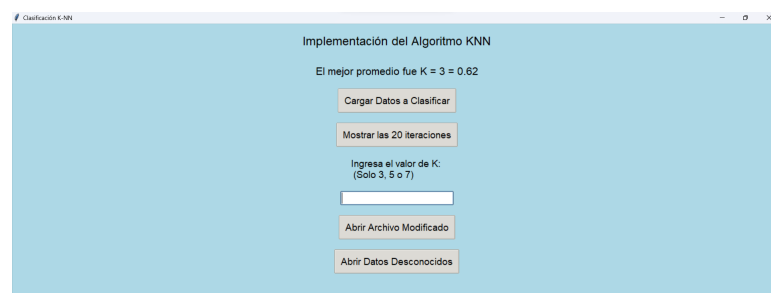


Figura 11: Ventana principal.

6. Explicación de la interfaz.

- El botón 'Cargar Datos a Clasificar' selecciona los datos desconocidos que queremos clasificar, en este caso será el archivo Desconocidos .csv.

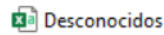


Figura 12: Archivo a cargar.

- Una vez seleccionado el archivo, llenamos el espacio en blanco cómo lo indica la Figura 14, una vez hecho esto nos mostrara la ventana de clasificación.

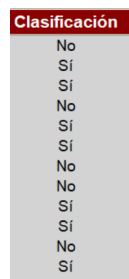
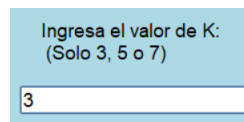


Figura 13: Ventana de clasificación.

- El botón 'Mostrar las 20 iteraciones' como su nombre lo indica muestra las 20 iteraciones evaluadas para K=3, K=5 y K=7.

20 Iteraciones y Resultados Promedio de Rendimiento				Resultados promedio de rendimiento: K = 3: 0.52 K = 5: 0.55 K = 7: 0.57
i	K = 3	K = 5	K = 7	
1	0.60	0.60	0.60	
2	0.40	0.40	0.40	
3	0.60	0.60	0.60	
4	0.60	1.00	0.80	
5	0.40	0.40	0.40	
6	0.40	0.40	0.40	
7	0.60	0.60	0.80	
8	0.60	0.60	0.60	
9	0.40	0.60	0.60	
10	0.60	0.60	0.60	
11	0.60	0.60	0.60	
12	0.60	0.60	0.60	
13	0.40	0.20	0.60	
14	0.60	0.60	0.60	

Figura 14: 20 iteraciones.

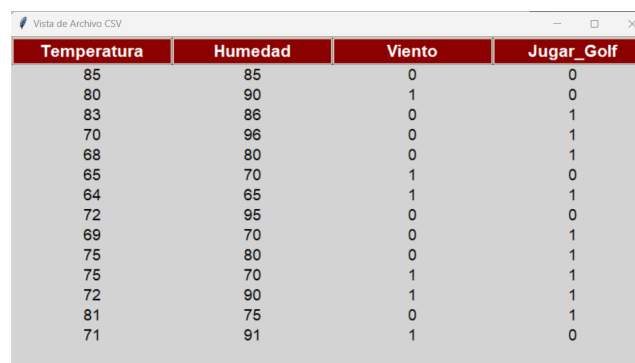


Ingresa el valor de K:
(Solo 3, 5 o 7)

3

Figura 15: Ingresar valor para K.

- En el espacio blanco ingresamos el valor de K para clasificar los datos desconocidos.
- El botón 'Abrir Archivo Modificado' nos muestra los datos que tenemos en el archivo .csv en este caso son los datos para engtrenar el modelo.



Temperatura	Humedad	Viento	Jugar_Golf
85	85	0	0
80	90	1	0
83	86	0	1
70	96	0	1
68	80	0	1
65	70	1	0
64	65	1	1
72	95	0	0
69	70	0	1
75	80	0	1
75	70	1	1
72	90	1	1
81	75	0	1
71	91	1	0

Figura 16: Archivo modificado.

- El botón 'Abrir Datos Desconocidos' nos muestra los datos desconocidos para clasificar.



Temperatura	Humedad	Viento	Jugar_Golf
77	92	0	0
63	78	1	0
88	70	0	1
72	95	0	1
79	84	1	0
68	72	0	1
85	88	0	0
74	90	1	0
62	75	0	1
81	82	1	0
73	89	0	1
67	76	0	1

Figura 17: Archivo desconocido.

6. Soporte y Ayuda Adicional.

Si necesita ayuda adicional o tiene preguntas sobre el código, no dude en consultar la documentación del código o ponerse en contacto con los desarrolladores del código.

7.4. DETALLES TÉCNICOS:

● Parte 1: Instalar Python

Para instalar Python en tu sistema Windows, sigue estos pasos:

1. **Descargar el instalador de Python:** Ve al sitio web oficial de Python (<https://www.python.org/downloads/>) y descarga el instalador adecuado para tu sistema Windows. Se recomienda descargar la versión más reciente de Python, que esté marcada como estable.
2. **Ejecutar el instalador:** Una vez que se haya completado la descarga, haz doble clic en el archivo descargado para ejecutar el instalador de Python. Asegúrate de marcar la opción «Agregar Python X.X a PATH» durante la instalación para que Python se agregue automáticamente a la variable de entorno PATH del sistema.
3. **Completar la instalación:** Sigue las instrucciones en pantalla para completar la instalación de Python. Asegúrate de marcar la opción para instalar pip, que es un administrador de paquetes de Python utilizado para instalar y administrar bibliotecas y paquetes de terceros.
4. **Verificar la instalación:** Una vez que la instalación haya finalizado, abre una ventana de comandos (CMD) y escribe el siguiente comando para verificar que Python se haya instalado correctamente:

```
python --version
```

Deberías ver la versión de Python instalada en tu sistema.

5. **Instalar un entorno de desarrollo (opcional):** Si planeas desarrollar en Python, considera instalar un entorno de desarrollo integrado (IDE) como Spyder, PyCharm o Visual Studio Code. Estos IDE proporcionan características adicionales, como resaltado de sintaxis, depuración de código y administración de proyectos.

¡Ahora has instalado Python en tu sistema Windows y estás listo para empezar a programar en Python!

● Parte 2: Anaconda

El programa que vamos a usar es muy fácil de usar e instalar, es un código abierto multiplataforma entorno de desarrollo integrado (IDE) para la programación científica en el Lenguaje python. Spyder se integra con una serie de paquetes prominentes en la pila científica de Python, que incluyen NumPy, SciPy, Matplotlib, pandas, IPython, Simpático y Citón, así como otro software de código abierto.

Instalación en Windows

Para instalar Spyder con Anaconda en Windows, siga estos pasos:

1. Descargue el instalador de Anaconda desde el sitio web oficial: <https://www.anaconda.com/products/distribution> guardelo en una carpeta conocida para ejecutar una vez terminada la descarga.
2. Ejecute el instalador descargado desde la carpeta en la que lo guardo y siga las instrucciones en pantalla para completar la instalación de Anaconda.
3. Una vez que Anaconda esté instalado, abra Anaconda Navigator desde el menú de inicio.
4. En Anaconda Navigator, haga clic en el botón 'Instalar' junto al paquete de Spyder.
5. Espere a que se complete la instalación y luego ejecute Spyder desde Anaconda Navigator o desde el menú de inicio.

● Parte 3: Spyder

1. **Explorar la interfaz de usuario:** Al abrir Spyder, verás una interfaz de usuario dividida en diferentes paneles, como el editor de código, el explorador de variables y la consola IPython. Familiarízate con estos paneles, ya que serán útiles para tu trabajo con Python.

2. **Escribir y ejecutar código:** Utiliza el editor de código para escribir tu código Python. Selecciona el código que deseas ejecutar y haz clic en el botón 'Ejecutar' en la barra de herramientas o presiona 'Ctrl + Enter'. Verás la salida de tu código en la consola IPython.
3. **Explorar características adicionales:** Spyder ofrece muchas características adicionales, como depuración de código, autocompletado de código y ayuda integrada. Explora estas características para mejorar tu productividad en el desarrollo de Python.

Referencias

- [1] Rodríguez, J. E. R., Blanco, E. A. R., & Camacho, R. O. F. (2007). Clasificación de datos usando el método k-nn. *Revista Vínculos*, 4(1), 4-18.
- [2] Villalba, L. M. M., Camacho, F. J. Á., Ramírez, A. M., & Stein, J. M. (2017). Modelo para la detección de melanoma mediante el escaneo de imágenes con Deep Learning usando el algoritmo K-NN (Abril 2018). *TecnoCultura*, 8-8.
- [3] Bobadilla, J. (2021). *Machine Learning y Deep Learning: usando Python, Scikit y Keras*. Ediciones de la U.